

Great — now I'll give you a clean, interview-ready, implementation-level explanation of NIYOJAN V3 end-to-end.

This is not just description — this is how you should think, explain, and build it.

NIYOJAN V3 — End-to-End System (Final Understanding)

1. What NIYOJAN V3 Actually Is

NIYOJAN V3 is:

An Agentic AI Decision Intelligence System built on top of a forecasting engine

- 👉 It does NOT predict
 - 👉 It does NOT train models
 - 👉 It transforms forecasts → decisions
-

Core Philosophy (VERY IMPORTANT)

“Prediction is not useful unless it leads to action”

So NIYOJAN V3 answers:

- What should I do?
 - What is risky?
 - What happens if things change?
-

2. Full System Architecture (Big Picture)

User → Forecast Module → Stored Outputs



Agentic AI Hub (V3)



Intent → Routing → Execution Paths



Context Aggregation



Strategic LLM Reasoning



Executive Output



3. User Flow (From UI Perspective)

Step 1 — Login

User lands on dashboard:

- Forecasting Engine
- Agentic AI Hub

Step 2 — Forecasting Module

User:

- uploads historical data

System:

- runs LSTM
- generates:
 - forecast CSV
 - report PDF

Stored in backend

Step 3 — Agentic AI Hub (Chat Interface)

User:

- selects project OR uploads:
 - forecast CSV
 - report PDF

Then asks:

- 👉 “What should I do next month?”
 - 👉 “What if demand increases by 15%?”
-

4. Data Layer (VERY CRITICAL)

Raw Inputs:

- Forecast CSV
 - Report PDF
-

Preprocessing Layer (MANDATORY)

Transforms raw data into:

```
[  
  {  
    product_id,  
    forecast_demand,  
    std_dev,  
    lead_time,  
    current_stock,  
    volatility,  
    reorder_point,  
    gap,  
    risk_score,  
    alert_from_report
```

```
}
```

```
]
```

Key Computations:

Volatility

$\text{std_dev} / \text{mean_demand}$

Safety Stock

$Z * \text{std_dev} * \sqrt{\text{lead_time}}$

Reorder Point

$(\text{demand} * \text{lead_time}) + \text{safety_stock}$

Inventory Gap

$\text{ROP} - \text{current_stock}$

👉 This layer ensures:

- ✓ deterministic logic
- ✓ no hallucination
- ✓ fast execution



5. Agentic AI Hub (Core Engine)

Built using LangGraph (single orchestrated agent)

◆ Agent State

```
class AgentState:
```

```
    query
```

```
    intent
```

```
    forecast_data
```

```
    rag_context
```

```
    analysis_result
```

```
    simulation_result
```

```
    strategic_context
```

```
    final_output
```



6. Step-by-Step Execution Flow



Step 1 — Intent Detection

Input:

```
"What if demand increases by 15%?"
```

Output:

```
intent = "simulation"
```

```
simulation_percent = 15
```



Step 2 — Conditional Routing

Based on intent:

Intent

Path

retrieval RAG

analysis metrics

simulation scenario
engine

strategy planning



7. Execution Paths (CORE LOGIC)

◆ Path 1 — Retrieval (RAG)

Used when:

- user asks explanation
- refers to report

Flow:

- embed query
- vector search
- retrieve chunks

👉 No calculations

◆ Path 2 — Analysis (Deterministic)

Compute:

- highest volatility SKU
- biggest gap
- lead time insights

👉 Pure Python

👉 No LLM

◆ Path 3 — Simulation (Most Important)

Example:

“What if demand increases by 15%?”

Steps:

1. Adjust Demand

$\text{new_demand} = \text{forecast} * 1.15$

2. Recompute Safety Stock

3. Recompute ROP

4. Compute Gap

5. Assign Risk

Gap > 200 → High

Gap 100–200 → Medium

Else → Low

👉 Fully deterministic

👉 No hallucination

◆ Path 4 — Strategy Preparation

This is bridge between data and LLM

Aggregates:

- high-risk SKUs
- volatility
- lead time
- simulation output
- report insights

Example Output:

```
{  
  "top_risks": [...],  
  "key_drivers": [...],  
  "priority_skus": [...]  
}
```



8. Strategic LLM Reasoning (ONLY LLM USAGE)

Input:

- structured context (NOT raw data)

LLM generates:

- procurement strategy
- prioritization
- action plan

👉 LLM does:

- ✓ reasoning
- ✗ NOT math



9. Context Aggregation

Combine:

- analysis results
 - simulation results
 - RAG context
-



10. Executive Formatter

Final output structured as:



Executive Summary

What is happening



High-Risk Products

Top SKUs with issues



Key Risk Drivers

Why risk exists



Recommended Actions

What to do



4-Week Action Plan

Week-wise actions



Monitoring Strategy

What to track



11. Final Output

User gets:

- ✓ Decision-ready insights
- ✓ Not just numbers
- ✓ Not just explanation



12. Backend Architecture

backend/

|

|— agents/

| |— graph.py

| |— state.py

| |— intent.py

| |— nodes/

| |— retrieval.py

| |— analysis.py

| |— simulation.py

| |— strategy.py

| |— reasoning.py

| |— formatter.py

|

|— tools/

| |— inventory_math.py

| |— risk_metrics.py

|

|— rag/

| |— ingestion.py

| |— vector_store.py

|

|— services/

| |— forecast_service.py

|

— routes/

— intelligence.py

13. Why This System Is Strong

✓ Deterministic Core

No hallucinated calculations

✓ Controlled LLM Usage

LLM only for reasoning

✓ Modular Design

Each node independent

✓ Explainable AI

Every output traceable

✓ Real Business Value

Not academic — usable

14. Limitations (Be Honest)

- static thresholds
 - no multi-product optimization
 - no real-time data
 - no learning loop
-



15. Future Scope

- adaptive thresholds
 - RL-based planning
 - real-time pipelines
 - optimization engine
-



FINAL ONE-LINER (INTERVIEW GOLD)

“NIYOJAN V3 is an agentic AI decision intelligence system that transforms demand forecasts into actionable procurement strategies using deterministic simulation, structured reasoning, and controlled LLM orchestration.”



Done — You Now Fully Understand V3

If you want next:

👉 I can now:

- convert this into interview answer script
- OR start full backend implementation
- OR write LangGraph code from scratch

Just tell me 👍